

# Banglalink Combined Network Measurement SDK Migration Guide

## Unified Network Measurement SDK to Crowdsourcing SDK and FWA SDK

Migration from the old Banglalink unified Network Measurement SDK to the two dedicated SDKs:

- Banglalink Crowdsourcing Network Measurement SDK for event-based background network measurement.
- Banglalink FWA Network Measurement SDK for FWA diagnostic measurement.

### 1. Objective

This document outlines migration steps for host applications that use both old unified SDK flows and now need to integrate both separated SDKs.

### 2. Migration Scope

The old unified SDK supported two capture types through one class:

#### CODE BLOCK - KOTLIN

```
NetworkDataUploader().startUploading(  
    ...,  
    uploadType = UploadType.NetworkDataCapture  
)  
  
NetworkDataUploader().startUploading(  
    ...,  
    uploadType = UploadType.FTPNetworkDataCapture  
)
```

In the new separated SDK setup:

| Old unified SDK flow             | New SDK           | New entry point                  |
|----------------------------------|-------------------|----------------------------------|
| UploadType.NetworkDataCapture    | Crowdsourcing SDK | NetworkCrowdSourcingDataUploader |
| UploadType.FTPNetworkDataCapture | FWA SDK           | FWANetworkDataMeasurement        |

Do not migrate event-based background measurement into the FWA SDK. Do not migrate FWA diagnostic measurement into the Crowdsourcing SDK.

## 3. Dependency Migration

### Old Unified SDK

In your app-level `build.gradle.kts`:

#### CODE BLOCK - KOTLIN

```
dependencies {  
  
    implementation("com.github.primetechsoltutions:network-measurement-unified-sdk:X.X.X")  
}
```

### New Separated SDKs

Add both SDKs when the host app needs both event-based crowdsourcing and FWA diagnostic measurement.

#### CODE BLOCK - KOTLIN

```
dependencies {  
  
    implementation("com.github.primetechsoltutions:network-measurement-crowdsourcing-sdk:X.X.X")  
  
    implementation("com.github.primetechsoltutions:FWA_Network_Measurement_SDK:X.X.X")  
}
```

Replace `X.X.X` with the approved production release tags.

## 4. Repository Migration

Keep JitPack in `settings.gradle.kts` or project-level repository configuration.

#### CODE BLOCK - KOTLIN

```
maven {  
    url = uri("https://jitpack.io")  
}
```

If the old unified SDK used a local Maven repository, remove it when it is no longer needed.

## 5. Import Migration

### Old Unified SDK

#### CODE BLOCK - KOTLIN

```
import com.ptsl.network_sdk.NetworkDataUploader
import com.ptsl.network_sdk.UploadType
```

### New Separated SDKs

#### CODE BLOCK - KOTLIN

```
import com.ptsl.crowdsourcing_network_sdk.NetworkCrowdSourcingDataUploader
import com.ptsl.fwa_network_sdk.FWANetworkDataMeasurement
```

UploadType is no longer needed. The SDK class now determines the measurement type.

## 6. Initialization Migration

### Old Unified SDK

#### CODE BLOCK - KOTLIN

```
class MainActivity : AppCompatActivity() {

    private val networkDataUploader = NetworkDataUploader()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        networkDataUploader.init(this, "MyBL")
    }
}
```

### New Combined SDK Setup

Initialize both SDKs if the same host app screen or application flow needs both features.

#### CODE BLOCK - KOTLIN

```
class MainActivity : AppCompatActivity() {

    private val crowdSdk = NetworkCrowdSourcingDataUploader()
    private val fwaSdk = FWANetworkDataMeasurement()

    override fun onCreate(savedInstanceState: Bundle?) {
```

```

        super.onCreate(savedInstanceState)

        crowdSdk.init(this, "MyBL")
        fwaSdk.init(this, "MyBL")
    }
}

```

Initialization should happen before calling either SDK measurement method.

## 7. Event-Based Crowdsourcing Migration

### Old Unified SDK

#### CODE BLOCK - KOTLIN

```

networkDataUploader.startUploading(
    msisdn = "01XXXXXXXXXX",
    integratedAppVersion = "1.0.0",
    sdkInitiateTimeStamp = "2026-07-02T10:30:00",
    integratedAppEventName = "Home_Page_Visit",
    uploadType = UploadType.NetworkDataCapture
) { success, status ->
    Log.d("SDK_RESPONSE", "Crowdsourcing: $success, ${status.response}")
}

```

### New Crowdsourcing SDK

#### CODE BLOCK - KOTLIN

```

crowdSdk.startCrowdSourcingUploading(
    msisdn = "01XXXXXXXXXX",
    integratedAppVersion = "1.0.0",
    sdkInitiateTimeStamp = "2026-07-02T10:30:00",
    integratedAppEventName = "Home_Page_Visit"
) { success, status ->
    Log.d("CROWD_SDK_RESPONSE", "Crowdsourcing: $success,
    ${status.response}")
}

```

Crowdsourcing measurement is event-based and runs as background network data capture.

## 8. FWA Diagnostic Migration

### Old Unified SDK

#### CODE BLOCK - KOTLIN

```
networkDataUploader.startUploading(  
    msisdn = "01XXXXXXXXXX",  
    integratedAppVersion = "1.0.0",  
    sdkInitiateTimeStamp = "2026-07-02T10:30:00",  
    integratedAppEventName = "Manual_Diagnostic",  
    uploadType = UploadType.FTPNetworkDataCapture  
) { success, status ->  
    Log.d("SDK_RESPONSE", "FWA: $success, ${status.response}")  
}
```

### New FWA SDK

#### CODE BLOCK - KOTLIN

```
fwaSdk.startFWAMeasurement(  
    msisdn = "01XXXXXXXXXX",  
    integratedAppVersion = "1.0.0",  
    sdkInitiateTimeStamp = "2026-07-02T10:30:00",  
    integratedAppEventName = "Manual_Diagnostic"  
) { success, status ->  
    Log.d("FWA_SDK_RESPONSE", "FWA: $success, ${status.response}")  
}
```

FWA measurement runs the diagnostic assessment, uploads the result, and returns the final assessment response to the host app.

## 9. API Mapping

| Old unified SDK               | New Crowdsourcing SDK              | New FWA SDK               |
|-------------------------------|------------------------------------|---------------------------|
| NetworkDataUploader           | NetworkCrowdSourcingDataUploader   | FWANetworkDataMeasurement |
| com.ptsl.network_sdk          | com.ptsl.crowdsourcing_network_sdk | com.ptsl.fwa_network_sdk  |
| startUploading(...)           | startCrowdSourcingUploading(...)   | startFWAMeasurement(...)  |
| UploadType.NetworkDataCapture | Use this SDK                       | Not supported             |

|                                   |                            |                      |
|-----------------------------------|----------------------------|----------------------|
| UploadType.FTPNetworkData Capture | Not supported              | Use this SDK         |
| UploadStatus                      | Crowdsourcing status model | FWAMeasurementStatus |

## 10. Parameter Mapping

| Old parameter          | Crowdsourcing SDK      | FWA SDK                | Notes                           |
|------------------------|------------------------|------------------------|---------------------------------|
| msisdn                 | msisdn                 | msisdn                 | Same meaning.                   |
| integratedAppVersion   | integratedAppVersion   | integratedAppVersion   | Same meaning.                   |
| sdkInitiateTimeStamp   | sdkInitiateTimeStamp   | sdkInitiateTimeStamp   | Use yyyy-MM-dd'T'HH:mm:ss.      |
| integratedAppEventName | integratedAppEventName | integratedAppEventName | Same meaning.                   |
| userLatitude           | userLatitude if used   | userLatitude if used   | Optional fallback, default 0.0. |
| userLongitude          | userLongitude if used  | userLongitude if used  | Optional fallback, default 0.0. |
| uploadType             | Removed                | Removed                | Choose SDK class instead.       |

## 11. Response Migration

### Crowdsourcing Response

Crowdsourcing responses are returned through the callback status object. A successful enqueue response is:

#### CODE BLOCK - JSON

```
{
  "status": "Success",
  "statusCode": 200,
  "message": "SDK Task enqueued"
}
```

If the SDK is not initialized:

#### CODE BLOCK - JSON

```
{
  "status": "Failed",
  "statusCode": 400,
  "message": "SDK not initialized"
}
```

## FWA Response

FWA responses are returned through `FWAMeasurementStatus.response`.

#### CODE BLOCK - KOTLIN

```
data class FWAMeasurementStatus(
    val isSdkInit: Boolean? = null,
    val isLocationEnabled: Boolean? = null,
    val isPhoneStateGranted: Boolean? = null,
    val response: String? = null
)
```

Success/pass response example:

#### CODE BLOCK - JSON

```
{
  "status": "Success",
  "testResult": "Pass",
  "statusCode": 200,
  "message": "Your network assessment successful.",
  "data": {
    "assessmentId": 119,
    "networkData": {
      "RSRP": -105,
      "SNR": 18,
      "RSRQ": -3
    },
    "cellInfo": {
      "cellName": "DHK_K1469_1",
      "eNodeBName": "DHK_K1469",
      "nbhDlThroughputMbps": 11.5,
      "nbhTrafficGB": 0.5
    },
    "speedPair": {
      "ulSpeedKbps": 5000.0,
      "dlSpeedKbps": 5000.0
    },
    "userInfo": {
      "msisdn": "01XXXXXXXXX",
      "deviceManufacture": "OnePlus",

```

```

        "deviceModel": "ONEPLUSA6000",
        "deviceOsVersion": "30",
        "latitude": 23.7515,
        "longitude": 90.3933
    }
}
}

```

## 12. Manifest and WorkManager Migration

If this block was added only for the old unified SDK, remove it.

### CODE BLOCK - XML

```

<provider
    android:name="androidx.startup.InitializationProvider"
    android:authorities="${applicationId}.androidx-startup"
    tools:node="remove" />

```

If the host app still disables AndroidX Startup for another feature, keep the host app's custom WorkManager initialization in the `Application` class.

## 13. Removing Old Unified SDK Usage

After migrating both flows:

- Remove old `com.ptsl.network_sdk.*` imports.
- Remove all `UploadType` references.
- Remove old unified SDK dependency.
- Keep both new SDK dependencies only if the host app uses both features.
- Remove SDK-originated Hilt configuration if it was added only for the old SDK.
- Remove old local Maven SDK repository if it was used only for the old SDK.

## 14. Support & Integration Sync

If needed, for better understanding regarding the combined SDK integration, please arrange a meeting with the SDK Development Team. The team can review both migrated flows and validate production readiness before release.